

CSC789 - AI Capstone Project Proposal

Title: Ontology-Based Scope Control for Conversational Agents: A Comparison with Prose and Blocklist Baselines

Project track: Track A: Research

Student name: Sravya Daggumalli (individual)

Problem and Motivation

Today it is common practice to tell LLM-based conversational agents what they may respond to through a free-text system prompt, and those instructions are usually a list of things the agent must not do. I ran into three problems with this while building agent guardrails on Azure AI Foundry for a personal project. There is no good way to check a prose prompt for contradictions or gaps except by trying questions one at a time. A blocklist can miss related cases its author did not anticipate prior. And permissions do not inherit: allowing a broad category does not reliably allow the narrower cases inside it unless each one is spelled out. Vendor guardrails cover fixed harm categories like hate or self-harm, not what a specific agent is supposed to be *for*.

This project analyses whether a structured scope specification, an ontology where permissions are attached to concepts and a permission on a parent concept carries down to its children, makes better allow/refuse decisions than a prose prompt or a blocklist, and what it costs to build it.

Related Work/Background

There is a lot of recent work on runtime policy enforcement for LLM agents, including AgentSpec [1], FORGE [2], PCAS [3], Agent Behavioral Contracts [4], and a recent paper on what runtime gates can enforce [5]. Most of it focuses on which tools an agent may call rather than what it may talk about, and in the papers I reviewed none uses description logic or inheritance over a domain hierarchy. Ontology-based agent policy was tried in the 2000s (KAoS [6], Rei [7]) but has not been revisited for LLMs. NeMo Guardrails [8] comes closest with topical rails, but has no hierarchy. I did not find an empirical comparison of hierarchical scope specification against prose for conversational scope, which is the gap this project aims to fill.

Research Questions

- Q1.** Does an ontology with inherited permissions let fewer out-of-scope questions through than a prose prompt, a blocklist, or a flat topic list?
- Q2.** Does it do that without wrongly refusing more in-scope questions?
- Q3.** When it fails, is the failure in matching the question to a concept or in the policy itself?

Project Challenges

The hardest part is matching a user's question to the right concept in the ontology. If the match is wrong, the reasoning on top of it cannot help, so I will measure this by running the ontology once with automatic matching and once with the correct concept supplied from my labels. When nothing matches, the system refuses, which is right for out-of-scope questions but wrong for in-scope ones, and I will report how often that happens. The prose and blocklist baselines need to be strong for the comparison to mean anything, so I will tune them on a small held-out set before the test set is fixed. Finally, the Dallas export I have covers only one department, so I plan to pull the full request-type list through an aggregate query on the open data portal.

Dataset

I will use the Dallas 311 Service Requests open data [10] in summary form only: a grouped query gives the departments and roughly 400 request types with their counts, and I build the ontology from that list. No individual records, addresses, or locations are used. Since the data only shows what the city handles, I plan to write the out-of-scope cases myself (emergencies, private property disputes, legal advice, other cities) from the public 311 help pages.

For testing I will build 160 questions in four groups of 40: questions that name a service directly, questions about a specific service where only its parent category was permitted, reworded or informal versions of those, and questions that should be refused. I will draft candidates with an LLM from the request-type list (without showing it the ontology), then review, edit, and label each one by hand with allow/refuse and the concept it belongs to. Someone not involved with the project will label 40 of them independently so I can report the agreement. From each group, 10 questions go to tuning and 30 to the test set, which is fixed before any results are checked. The scope model is a dummy city, not Dallas's actual policy.

Proposed AI Approach

All approaches take a question and return **allow** or **refuse** with a short reason. No chat interface or answer text is built and the decision is used as a measure.

- **(A) Prose prompt.** A system prompt describing what the assistant covers, run through a small hosted LLM.
- **(B) Prose + blacklist.** Same prompt plus an explicit list of forbidden topics.
- **(R) Flat list.** The same concepts as the ontology, no hierarchy, matched by embedding similarity. No LLM.
- **(C) Ontology (proposed).** 60–100 concepts, three levels deep, built in Protégé from the Dallas request types, each marked permitted or denied.
 - Match the question to the closest concept with a small sentence-embedding model (e.g. `bge-small-en-v1.5`); no match above threshold means refuse.
 - Walk the concept’s ancestors in Python with a most-specific-wins rule, so a parent’s permission covers its children unless a child overrides it.
 - An OWL reasoner via `owlready2` [9] keeps the hierarchy consistent and flags any concept marked both permitted and denied.
- **(D) Ontology with gold concept.** Same as C but the correct concept comes from my labels, skipping matching. A diagnostic upper bound, not a usable system.

Evaluation Plan

- **Metrics,** per approach and per question group on the 120 test questions: out-of-scope questions allowed; in-scope questions refused; for the ontology, how often a question matched any concept and how often an acceptable one.
- **Comparison:** McNemar’s test between approaches (all see the same questions), plus a table of which questions each approach got wrong.
- **Qualitative:** the inheritance path used for questions that depend on it; the reasoner catching a concept marked both permitted and denied before any question is run.
- **Expected result:** the ontology does best on inheritance-dependent questions, loses some on reworded questions to matching errors, and differs little on questions that name a service directly.
- **Schedule:** taxonomy, questions, and ontology by Week 6; runs and analysis Weeks 7-9; and a report after that. API should cost under \$30; everything else runs on my computer.

Expected Contribution / Value

A small, reproducible comparison of prose, blacklist, flat-list, and hierarchical scope policies on a constructed 311 benchmark, produced with the question set and ontology. It will show whether inheritance helps, how much is lost to automatic matching, and that a structured policy can be checked for conflicts before use. If a well-tuned prompt matches the ontology at this scale, that is still a useful result about when a formal specification is worth the effort.

References

- [1] H. Wang, C. M. Poskitt, and J. Sun. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. *Proc. ICSE 2026*. arXiv:2503.18666.
- [2] Formal Policy Enforcement for Real-World Agentic Systems (FORGE). arXiv:2602.16708, 2026.
- [3] PCAS: Policy Compiler for Secure Agentic Systems. arXiv, 2026.
- [4] V. P. Bhardwaj. Agent Behavioral Contracts: Formal Specification and Runtime Enforcement for Reliable Autonomous AI Agents. arXiv:2602.22302, 2026.
- [5] S. Ray. What Can Be Enforced? A Theory of Certified Runtime Safety for Tool-Using Agents. arXiv:2607.22868, 2026.
- [6] A. Uszok, J. M. Bradshaw, et al. KAoS Policy and Domain Services: Toward a Description-Logic Approach to Policy Representation, Deconfliction, and Enforcement. *IEEE POLICY 2003*.
- [7] L. Kagal, T. Finin, and A. Joshi. A Policy Language for a Pervasive Computing Environment. *IEEE POLICY 2003*.
- [8] T. Rebedea et al. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. *EMNLP 2023 System Demonstrations*.
- [9] J.-B. Lamy. Owlready: Ontology-Oriented Programming in Python with Automatic Classification and High Level Constructs for Biomedical Ontologies. *Artificial Intelligence in Medicine* 80:11–28, 2017.
- [10] City of Dallas. 311 Service Requests, October 1, 2000 to Present [dataset]. Dallas OpenData, id 4n3h-rfqm. <https://www.dallasopendata.com/Services/311-Service-Requests-October-1-2000-to-Present/4n3h-rfqm>